
SWEG 3108

Advanced Programming: Java Packages

Compiled by: merid.nigussie@aastu.edu.et

Source: <http://www.cs.washington.edu/331/>

Diagnostic Test #1

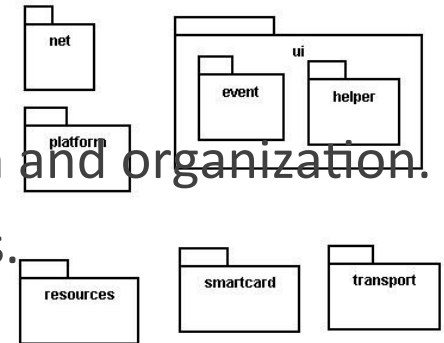
1. List basic principles of object oriented programming paradigm
2. List java access modifiers
3. Import all member classes of package named “student” using java in a single line
4. List top java packages (APIs) you know
5. List types of c++ functions

Cont... (True/False)

1. A variable allows storing a data value in the memory of a computer
2. A variable in programming is similar to a mathematical variable
3. A variable is a portion of code in a program
4. A fragement of code “c++;” is statement
5. Variable declation is similar to initialization

Java packages

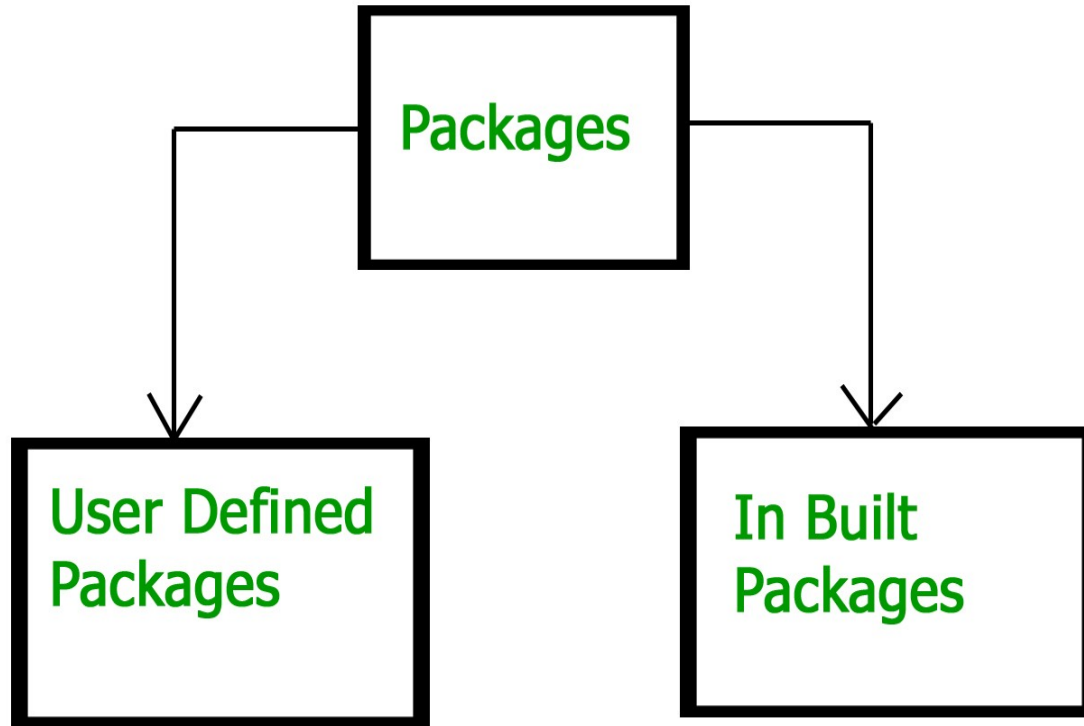
- **Package:** A collection of related classes.
 - Can also "contain" sub-packages.
 - Packages are Java's way of doing large-scale design and organization.
 - They are used both to categorize and group classes.
- Uses of Java packages:
 - Group related classes together as a *namespace* to avoid name collisions
 - Provide a layer of access / protection
 - keep pieces of a project down to a manageable size



Packages are used

- Preventing naming conflicts.
- For example there can be two classes with name Employee in two packages, college.staff.se.Employee and college.staff.ee.Employee
 - `import java.util.Date;`
 - `import my.package.Date;`
- Providing controlled access: protected and default have package level access control.
 - A protected member is accessible by classes in the same package and its subclasses.
 - A default member (without any access specifier) is accessible by classes in the same package only.
- Packages can be considered as data encapsulation (or data-hiding).
- Packages naming convention is in reverse order of domain names, i.e., et.edu.se.ceme.aastu

Package Types



JAVA SYSTEM PACKAGES & THEIR CLASSES

- **java.lang**

Language Support classes. These are classes that java compiler itself uses & therefore they are automatically imported. They include classes for primitive types, strings, maths function, threads & exception.

- **java.util**

Language Utility classes such as vector, hash tables, random numbers, date etc.

- **java.io**

Input /Output support classes. They provide facilities for the input & output of data

- **java.awt**

Set of classes for implementing graphical user interface. They include classes for windows, buttons, list, menus & so on.

- **java.net**

Classes for networking. They include classes for communicating with local computers as well as with internet servers.

- **java.applet**

Classes for creating & implementing applets.

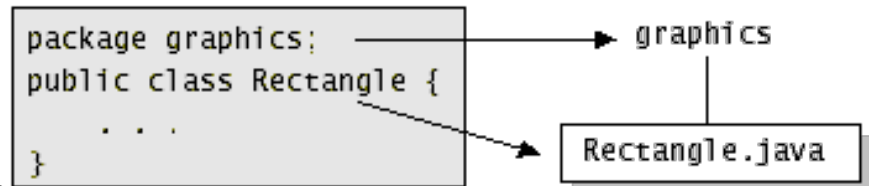


Packages and directories

- package $\leftrightarrow$ directory (folder)
- class $\leftrightarrow$ file
- A class named D in package a.b.c should reside in this file:

a/b/c/D.class

- (relative to the root of your project)



- The "root" directory of the package hierarchy is determined by your *class path* or the directory from which `java` was run.

Classpath

- **class path:** The location(s) in which Java looks for class files.
- Can include:
 - the current "working directory" from which you ran javac /java
 - other folders
 - JAR archives
 - URLs
 - ...
- Can set class path manually when running java at command line:
 - `java -cp /home/stepp/libs:/foo/bar/jbl MyClass`

A package declaration

```
package name;
```

```
public class name { ...
```

Example:

```
package person.model;
```

```
public class Student extends Person{  
    ...  
}
```

- File `Student.java` should go in folder `person/model/`.

Static import

```
import static packageName.className.*;
```

- Allows members (fields and methods) defined in a class as public static to be used in Java code without specifying the class in which the field is defined.

Example:

```
import static java.lang.Math.*;
```

```
double angle = sin(PI / 2) + ln(E * E);
```

- Static import allows you to refer to the **members of another class** without writing that class's name.
- Should be used rarely and only with classes whose contents are entirely static "utility" code.

Referring to packages

packageName . className

Example:

```
java.util.Scanner console = new  
    java.util.Scanner(java.lang.System.in);
```

- You can use a type from any package without importing it if you write its full name.
- Sometimes this is useful to disambiguate similar names.
 - Example: `java.awt.List` and `java.util.List`
 - Or, explicitly import one of the classes.

The default package

- Compilation units (files) that do not declare a package are put into a default, unnamed, package.
- Classes in the default package:
 - Cannot be imported
 - Cannot be used by classes in other packages
- Many editors discourage the use of the default package.
- Package `java.lang` is implicitly imported in all programs by default.
 - `import java.lang.*;`

Package access

- Java provides the following access modifiers:
 - `public` : Visible to all other classes.
 - `private` : Visible only to the current class (and any nested types).
 - `protected` : Visible to the current class, any of its subclasses, and any other types within the same package.
 - default (package): Visible to the current class and any other types within the same package.
- To give a member default scope, do not write a modifier:

```
package person.model;  
public class Student{  
    int points;           // visible to person.model.*  
    String name;          // visible to person.model.*  
}
```

Package exercise

- Add packages to the **“student-management-system”**.
Create a package for core "model" data.
Create a package for graphical "view" classes.
Create a package for business logic “controller” classes
- Any general utility code can go into a default package or into another named utility (util) package.
- Add appropriate package and import statements so that the types can use each other properly.

Demo

IDE